

Titolo del lavoro

Nome Cognome

Dipartimento / Università / Team

19 agosto 2026

Indice

1	Introduzione	3
1.1	Obiettivi	3
1.2	Metodologia in breve	3
1.3	Dataset Utilizzati	4
1.3.1	PhonItaliaR	4
1.3.2	Q2Stress	4
1.3.3	NRC	5
1.3.4	lakh-midi	5
2	Dal testo agli accenti al ritmo	6
2.1	Approccio generale	6
2.2	Analisi Prosodica	6
2.3	Librerie usate	6
2.3.1	re	6
2.3.2	UnicodeData	7
2.3.3	Pyphen	7
2.4	Algoritmo	7
2.4.1	Analisi prosodica	7
2.4.2	Produzione dello schema ritmico	9
2.4.3	Scelta degli strumenti	9
2.4.4	Produzione della melodia	9
2.5	Analisi emotiva	9
2.6	Librerie	9
2.6.1	re	9
2.7	Algoritmo	10

3	Selezione degli strumenti	11
3.1	Metodologie utilizzate	11
3.2	Transformer	11
3.3	Librerie	12
3.3.1	Pythorch	12
3.4	Trainig del modello	12
3.5	MIDI	13
4	Conclusioni	15
4.1	Valutazione finale	15
4.2	Sintesi	15
4.3	Sviluppi futuri	15
A	Appendice A	16

Capitolo 1

Introduzione

Questa introduzione mira a fornire a chi legge una panoramica degli obiettivi e delle metodologie utilizzate in questo progetto.

1.1 Obiettivi

Il progetto ha come obiettivo principale la definizione di una pipeline coerente per trasformare un input testuale in una rappresentazione musicale espressiva e controllabile. Nello specifico il progetto mira a raggiungere i seguenti obbiettivi:

1. Prendere in input un testo o file testuale e trovare secondo le regole della linua italiana le varie tipologie di accenti.
2. Tradurre le parole facendo anche uso degli accenti trovati in "ritmo" musicale.
3. Generare una melodia coerente con il ritmo precedentemente generato lasciando libertà di personalizzazione all'utente.

1.2 Metodologia in breve

Al fine di raggiungere gli obiettivi precedentemente presentati sarà seguita la seguente pipeline:

1. Analisi del testo che sarà svolta parallelamente dal punto di vista semantico e da quello prosodico (degli accenti). L'analisi semantica sarà effettuata per "assegnare" la musica corretta in base al significato e al contesto emotivo del testo. L'analisi prosodica sarà usata con il fine di trovare gli accenti, inoltre per una migliore definizione del ritmo sarà importante definire il numero di sillabe per parola.
2. Dopo aver svolto l'analisi definita nel punto 1, si procederà alla traduzione del "brano" ritmico ottenuto in una scala ritmica musicale, in modo da ottenere un ritmo coerente con il testo.

3. Generazione di una melodia coerente con il ritmo generato.

Lo strumento principale usato per la realizzazione di questo progetto sarà Python, in quanto dotato di numerose librerie che saranno presentate di volta in volta estremamente utili allo svolgimento del progetto. Inoltre l'addestramento dei modelli si farà uso di dataset reperibili online ricchi di informazioni utili.

1.3 Dataset Utilizzati

Nell'ambito del machine learning e del deep learning come è ben noto per l'apprendimento del modello creato è necessario dare uso di dataset di training e di test. Per questo progetto sono stati utilizzati i seguenti dataset:

1.3.1 PhonItaliaR

PhonItaliaR fornisce rappresentazioni fonologiche per 120.000 forme di parole italiane. Ognuno di questi include anche confini sillabici, marcature di accento e una gamma completa di statistiche lessicali. Utilizzando dati derivati da questo lessico, è anche possibile generare un insieme di database derivati e fornito stime dell'uso della frequenza posizionale per fonemi, sillabe, inizi e coda sillabiche italiane, nonché bigrammi di caratteri e fonemi.

Fonte:<https://link.springer.com/article/10.3758/s13428-013-0400-8>

In questo progetto il dataset PhonItaliaR è estremamente importante.

Sarà usato per individuare su quale sillaba cade l'accento di una parola. È annotato da linguisti su 120.000 forme, non dedotto da regole, è quindi la fonte più affidabile.

Inoltre verrà fatto uso di PhonItaliaR anche per il conteggio delle sillabe, l'integrazione più recente fornisce il numero "vero" di sillabe (SumSylls), usato per correggere la sillabazione quando l'euristica interna sbaglia (tipicamente sui casi di iato: "poesia", "farmacia").

1.3.2 Q2Stress

Q2Stress è un database per l'italiano che contiene informazioni su molteplici segnali che i lettori possono utilizzare per assegnare l'accento. In particolare, il database offre misure di frequenza di tipo e token dei pattern di accento in funzione del numero di sillabe, della categoria grammaticale, dell'inizio delle parole, delle terminazioni delle parole e delle strutture consonanti-vocali. Sono disponibili dati sia per adulti, sia per i bambini.

Q2Stress può aiutare i ricercatori a rispondere a domande empiriche e teoriche sull'assegnazione dell'accento e sulla relazione tra ortografia e fonologia nella lettura. Q2Stress è progettato come una risorsa facile da usare, poiché include script che permettono ai ricercatori di esplorare e selezionare i propri stimoli secondo diversi criteri, oltre a tabelle riassuntive per l'analisi complessiva dei dati. Fonte:<https://www.istc.cnr.it/en/content/q2stress>.

In questo progetto il dataset Q2Strss sarà usato in sinergia con PhonItaliaR per l'analisi fonetica. In particolare Q2Stress in questo progetto sarà un "ripiego", per quando PhonItalia non ha la parola (neologismi, forme rare, parole poetiche/arcaiche). Invece di ricadere subito sull'euristica grezza ("parola piana di default"). Q2Stress guarda la desinenza delle ultime 3 lettere e restituisce la posizione dell'accento più probabile secondo le statistiche reali sull'italiano (es. l'80% delle parole che finiscono in "-ino" sono piane).

1.3.3 NRC

1.3.4 lakh-midi

Il dataset MIDI di Lakh è una raccolta di 176.581 file MIDI unici, di cui 45.129 sono stati abbinati e allineati alle voci del Million Song Dataset. Il suo obiettivo è facilitare il recupero su larga scala delle informazioni musicali, sia simbolico (usando solo i file MIDI) sia basato su contenuti audio (utilizzando informazioni estratte dai file MIDI come annotazioni per i file audio abbinati).

Fonte: <https://colinraffel.com/projects/lmd/>

Nel contesto di questo progetto il dataset lakh-midi si occupa della parte della generazione musicale.

Capitolo 2

Dal testo agli accenti al ritmo

2.1 Approccio generale

In questo capitolo ci si concentrerà sulla prima parte del progetto, ovvero la lettura del testo dato in input e la successiva analisi prosodica e semantica. Dopo si procederà con l'individuazione delle sillabe e degli accenti. Successivamente sarà fatta anche un'analisi emotiva per attribuire una melodia consona con il significato del testo e infine sarà prodotto lo scheletro ritmico.

2.2 Analisi Prosodica

Con analisi prosodica si intende è la parte della linguistica che studia l'intonazione, il ritmo (isocronia), la durata (quantità) e l'accento del linguaggio parlato.

In questo progetto ovviamente il linguaggio di riferimento sarà l'Italiano e come spiegato nel precedente capitolo i dataset importati per tale analisi sono Q2Stress e PhonItaliaR.

La pipeline di lavoro per questa parte sarà la seguente: Si prenderà il verso, dal verso saranno estratte le sillabe da cui poi si troveranno pattern accentuativo e alla fine sarà estratto il ritmo.

2.3 Librerie usate

2.3.1 re

Questa libreria offre delle operazioni svolgibili sulle espressioni regolari.

Un'espressione regolare specifica un set di stringhe che combacia con essa. Le funzioni di questa libreria permettono di verificare se una particolare stringa combacia con una particolare espressione regolare. Le espressioni regolari possono essere concatenate per formare nuove espressioni regolari; se A e B sono entrambe espressioni regolari, allora AB è anch'essa un'espressione regolare. In generale, se una stringa p corrisponde ad A e un'altra stringa q corrisponde a B, la stringa pq corrisponderà ad AB. Questo vale a meno che A o B contengano

operazioni a bassa priorità; condizioni ai confini tra A e B; o riferimenti a gruppi numerati. Fonte:<https://docs.python.org/3/library/re.html>

Nel codice implementato la libreria `re` serve per usare la funzione `re.sub` che rimuove tutti i caratteri di punteggiatura da una parola mantenendo solo le lettere e i caratteri accentati.

2.3.2 UnicodeData

La libreria dà accesso all'Unicode Character Database (UCD) che definisce le proprietà dei caratteri di tutti i caratteri Unicode.

Fonte: <https://docs.python.org/3/library/unicodedata.html>

Viene importata per gestire correttamente la codifica dei caratteri speciali e accentati. In particolare viene usato il metodo `lookup` che cerca un carattere dal nome. Se un carattere con tale nome viene trovato ritorna il carattere corrispondente, altrimenti viene lanciato un `Keyerror`.

2.3.3 Pyphen

Libreria Python per la sillabazione basata sui dizionari e sugli algoritmi di sillabazione di TeX (come quelli usati da OpenOffice/LibreOffice).

Nel codice viene usata come meccanismo di ripiego (`fallback`) o di confronto per la sillabazione dell'italiano (`it_IT`).

2.4 Algoritmo

L'algoritmo esegue l'analisi prosodica e ritmica di un testo poetico italiano trasformando ogni verso in una sequenza di impulsi temporali. Il processo si articola in quattro fasi consecutive.

2.4.1 Analisi prosodica

Per analizzare la prosodia di un testo, l'algoritmo dedicato procede al caricamento di due dataset: `Q2Stress` e `phonItaliaR`. Il primo viene utilizzato per addestrare il classificatore di accenti, il secondo per ampliare il training set. Inoltre, viene caricato anche il modello linguistico `spaCy` per l'analisi linguistica del testo (serve a tokenizzare il testo?). Le informazioni estratte dai dataset per la costruzione delle feature sono le parole annotate, la posizione dell'accento e il numero di sillabe della parola. Per ogni parola vengono estratte le seguenti feature:

1. Lunghezza della parola
2. Numero di vocali
3. Numero di consonanti
4. Rapporto vocali/consonanti

5. Numero di sillabe
6. Finale vocalico
7. Finale consonantico
8. Presenza di sequenze vocaliche (...)
9. Presenza di sequenze con t (...)
10. Suffissi finali (...)

In tutto sono 24 e servono per il calcolo della posizione dell'accento tonico per ogni parola del testo di input.

Queste caratteristiche sono utilizzate per addestrare il classificatore **Random Forest**. È un modello di Machine Learning supervisionato basato sugli alberi decisionali. Dopo l'addestramento sui dataset, impara a predire la posizione dell'accento tonico della parola categorizzandola in una delle seguenti categorie:

1. ultima sillaba
2. penultima sillaba
3. antepenultima sillaba
4. preantepenultima sillaba

In seguito, si passa all'acquisizione e alla normalizzazione del testo. Qui vengono eliminati eventuali problemi di codifica, uniformati gli spazi e rimossi gli elementi non necessari all'analisi (...). Il testo viene poi elaborato tramite spaCy, il quale per ogni parola ottiene:

1. token
2. lemma
3. categoria grammaticale (POS)
4. relazioni sintattiche
5. posizione nella frase
6. punteggiatura circostante

Prima di passare al Random Forest, avviene il conteggio delle sillabe grazie ai dati annotati nei dataset, Pyphen e un algoritmo euristico di fallback (...).

Infine, tutte queste informazioni vengono trasformate in una sequenza di impulsi ritmici. Ogni sillaba viene classificata come:

1. strong: accentata, dove sarà posizionata la nota dominante (corrispondente a un hit di batteria?)
2. weak: non accentata, dove sarà posizionata la stessa nota più lieve (hat?)

Inoltre, la punteggiatura generale aggiunge pause brevi e lunghe.

Infine, il sistema produce:

1. CSV analitico, il quale per ogni parola contiene l'accento predetto, il numero di sillabe, informazioni grammaticali e sintattiche
2. CSV ritmico che contiene la sequenza di impulsi forti, deboli e pause.
3. schema ritmico MIDI (opzionale?) utilizzabile nelle seguenti fasi di generazione musicale.

2.4.2 Produzione dello schema ritmico

(forse si può approfondire... togliendo la parte finale della sezione di sopra)

2.4.3 Scelta degli strumenti

2.4.4 Produzione della melodia

2.5 Analisi emotiva

Come prima cosa viene importato un dataset che sarà utilizzato per definire lo spettro emotivo della parola.

2.6 Librerie

2.6.1 re

Anche in questo algoritmo viene usata la libreria `re` già descritta precedentemente. In particolare viene usata in una funzione che si occupa di normalizzare una parola, ovvero la trasforma minuscola, senza punteggiatura, ciò è necessario perchè prima di iniziare con l'algoritmo è necessario controllare che la parola sia formattata correttamente.

Per implementare questa funzione vengono usate le funzioni `.lower` che come deducibile dal nome rende tutte le lettere minuscole e `.sub` già incontrata in precedenza.

Inoltre viene usata la funzione `.findall` che permette di ottenere gruppi di stringhe con pattern simili per la tokenizzazione

2.7 Algoritmo

Il cuore dell'algoritmo si trova nella funzione `analyze_emotions` che non classifica le parole in semplici categorie (come "triste" o "felice"), ma le mappa su tre dimensioni psicologiche continue:

- Valence (Valenza): Indica se un'emozione è positiva o negativa. Va da -1.0 (estremamente negativo, es. abominio) a +1.0 (estremamente positivo, es. felicità).
- Arousal (Attivazione): Misura l'intensità o il livello di eccitazione dell'emozione. Va da 0.0 (calma/letargia, es. pace) a +1.0 (alta eccitazione/tensione, es. terrore o vivacissimo).
- Tenderness (Tenerezza): Misura il grado di dolcezza o affetto. I valori positivi indicano tenerezza (es. amore = 1.0), mentre i valori bassi o negativi indicano durezza o brutalità (es. brutale = -0.6).

La funzione procede nel seguente modo:

1. Viene eseguita la "Tokenizzazione", il testo in ingresso viene suddiviso in singole parole usando un'espressione regolare.
2. Viene eseguita l'estrazione dei punteggi per ogni parola. La parola viene normalizzata, l'algoritmo controlla se la parola normalizzata esiste nel database di riferimento, se c'è un match (corrispondenza), estrae i tre valori (valenza, attivazione, tenerezza) e li salva in tre liste separate. La parola stessa viene aggiunta a una lista se la parola non è nel dizionario (es. articoli, congiunzioni o parole non censite), viene semplicemente ignorata.
3. Viene effettuato il calcolo della media dopo aver analizzato tutte le parole, l'algoritmo calcola la media aritmetica dei punteggi trovati. Questo rappresenta il "tono emotivo generale" della frase. Se nessuna parola del testo è presente nel dizionario, l'algoritmo restituisce valori neutri ovvero valence 0.0 (neutro), arousal 0.5 (attivazione media) e tenderness: 0.0 (neutro).
4. Viene calcolato il risultato finale: Viene Restituito un dizionario contenente le medie calcolate e la lista delle parole riconosciute.

Capitolo 3

Selezione degli strumenti

3.1 Metodologie utilizzate

Una volta completata l'analisi prosodica ed emotiva si può passare alla parte di produzione musicale.

Per svolgere questo task si farà uso di un transformer.

3.2 Transformer

Il transformer è un modello di apprendimento profondo che adotta il meccanismo della auto-attenzione, pesando differentemente la significatività di ogni parte dei dati in ingresso.

Come le reti neurali ricorrenti (RNN), i trasformatori sono progettati per processare dati sequenziali come, ad esempio, testi in linguaggio naturale, con applicazioni alla loro generazione e traduzione automatica. Tuttavia, a differenza delle RNN, i trasformatori elaborano l'intero insieme di dati d'ingresso simultaneamente. Il cosiddetto meccanismo di attenzione fornisce il contesto per ogni posizione nella sequenza di ingresso. Per esempio, se i dati rappresentano una frase, il trasformatore non deve elaborare una parola alla volta: questo permette una maggiore parallelizzazione rispetto alle RNN e perciò porta a ridurre i tempi di addestramento.

Fonte: <https://it.wikipedia.org/wiki/Trasformatore>

Nell'ambito del progetto il transformer sarà usato per il fine della produzione musicale.

3.3 Librerie

3.3.1 Pythorch

3.4 Trainig del modello

L'obiettivo dell'addestramento è insegnare alla rete la grammatica e la sintassi melodica generale (come le note si susseguono in modo musicale) e come piegare questa melodia in base al contesto emotivo (Valenza e Arousal).

L'addestramento funziona nelle seguenti fasi:

1. Invece di insegnare alla rete le note assolute (es. Do, Re, Mi), il modello impara gli intervalli melodici in semitoni tra una nota e la successiva (es. +2 per un salto di tono in avanti, -3 per una terza minore all'indietro). Un salto di +2 semitoni ha la stessa "funzione melodica" sia in Do maggiore che in Fa# maggiore. Questo riduce il vocabolario a soli 25 token. La mappatura dei token funziona attraverso la seguente formula:

$$\text{Token ID} = \text{Intervallo (in semitoni)} + 12$$

2. Per evitare l'overfitting sul corpus MIDI (composto da poche migliaia di file) e rendere il modello capace di generalizzare su qualsiasi nuova poesia, durante l'addestramento vengono applicate due tecniche fondamentali:

Per ogni sequenza di intervalli $[s_1, s_2, s_3, \dots]$, lo script genera la sua versione speculare $[-s_1, -s_2, -s_3, \dots]$. Questo raddoppia la dimensione del dataset e insegna alla rete che i movimenti melodici speculari sono egualmente validi.

Nel 15% dei casi di addestramento, i valori di Valenza e Arousal passati alla rete vengono impostati forzatamente a zero $[0.0, 0.0]$. Si fa per Costringe il Transformer a imparare prima la struttura della sintassi musicale in modo indipendente ("incondizionato") e solo successivamente a correlarla alle emozioni, evitando che si affidi unicamente all'emozione ignorando la coerenza melodica.

3. Il modello elabora il contesto attraverso due flussi che si uniscono:

- Embedding Emotivo: La coppia di valori continui $(V, A) \in [-1, 1]$ passa attraverso uno strato denso (nn.Linear) che la proietta in un vettore a 128 dimensioni. Questo vettore viene concatenato all'inizio della sequenza come primo token (prefisso).
- Causal Masking (Maschera Triangolare Superiore): Durante l'addestramento, il Transformer vede l'intera sequenza contemporaneamente per calcolare il gradiente in parallelo. Per evitare che il modello "spii" le note future per predire quella attuale, si applica una maschera causale:

$$\text{Mask}_{ij} = \begin{cases} 0 & \text{se } i \geq j \quad (\text{può guardare solo al passato}) \\ -\infty & \text{se } i < j \quad (\text{bloccato}) \end{cases}$$

4. La rete viene addestrata con un approccio Self-Supervised (Autoregressivo): data una sequenza di intervalli fino al tempo t , la rete deve predire il token dell'intervallo al tempo $t + 1$.
5. L'input è la sequenza dal token 0 a $N - 1$, mentre il target da confrontare è la sequenza traslata da 1 a N . La Misura la distanza tra le probabilità calcolate dalla Softmax della rete e l'effettivo token di destinazione:

$$\mathcal{L} = - \sum_{c=0}^{24} y_c \log(\hat{y}_c)$$

Per l'ottimizzazione viene usato l'ottimizzatore AdamW con: Weight Decay (10^{-2}): Penalizza i pesi sinaptici che crescono troppo, impedendo alla rete di fossilizzarsi e il Dropout (0.2) che disattiva casualmente il 20% dei neuroni ad ogni passaggio per evitare la dipendenza tra singoli nodi.

Una volta completato il training sarà possibile effettuare la generazione della melodia usando il modello precedentemente creato.

1. A ogni passo, la rete calcola i logits per la nota successiva. Applicando una Temperatura ($T = 0.85$) e un Cut-off Top-p ($p = 0.90$), seleziona solo il sottoinsieme di intervalli più plausibili e ne campiona uno probabilisticamente.
2. Gli intervalli generati dal Transformer vengono infine "agganciati" ai gradi della scala selezionata dall'analisi prosodico-emotiva, garantendo un risultato privo di dissonanze.

è inoltre importante dire che il modello si affida unicamente alla maschera causale per l'informazione d'ordine, senza un positional encoding esplicito, scelta accettabile data la lunghezza contenuta delle sequenze.

3.5 MIDI

Una volta terminata la parte legata al transformer si potrà iniziare a fare uso di MIDI per la creazione del vero e proprio file musicale. Ciascuna nota viene rappresentata in memoria come un oggetto o tupla contenente:

- Pitch (Altezza): Numero intero compreso tra 0 e 127 (secondo lo standard MIDI, dove es. 60 = Do centrale / C4).
- Start Time (Tempo d'inizio): Espresso in battute o secondi assoluti dall'inizio del brano.
- Duration (Durata): Durata della nota espressa in frazioni di battuta o secondi.
- Velocity (Dinamica/Volume di attacco): Valore tra 1 e 127, derivato dall'Arousal emotivo (valori più alti per sezioni più intense/energetiche).

Utilizzando la libreria Python `mido`, viene creato il file MIDI Type 1 che supporta tracce multiple indipendenti che suonano in parallelo mantenendo un'unica sequenza temporale sincronizzata. Successivamente viene definita la risoluzione temporale, espressa in TPQN (Ticks Per Quarter Note), solitamente impostata su 480 o 960 tick per semiminima (quarto). Questo garantisce una griglia temporale precisa sia per le note in levare sia per gli arpeggi veloci.

Viene poi creata una prima traccia (o inseriti nell'header) i messaggi di meta-informazione:

- BPM / Tempo: Convertito nel valore microsecondi-per-quarto tramite `mido.bpm2tempo(bpm)`.
- Time Signature: Impostata la misura del brano (es. 4/4).

Successivamente il file viene suddiviso in 4 tracce MIDI distinte (MidiTrack), ciascuna associata a un canale MIDI indipendente (da 0 a 3):

- Track 0: la melodia principale prosodica.
- Track 1: Il tappeto armonico (accordi).
- Track 2: La linea di basso fondamentale.
- Track 3: Arpeggio e movimento.

I file MIDI non memorizzano il tempo assoluto (es. "suona a 2.5 secondi"), ma utilizzano il Delta-Time: il tempo trascorso tra l'evento precedente e quello attuale sulla medesima traccia. Per ogni traccia, tutti gli eventi di inizio nota (Note On) e fine nota (Note Off) vengono convertiti in coppie di eventi temporali e ordinati cronologicamente, il tempo in battute/secondi viene convertito in numero di tick:

$$\text{ticks} = \text{tempo_in_quarti} \times \text{TPQN}$$

Per ogni evento E_i al tempo T_i , il delta-time è dato da:

$$\Delta t = T_i - T_{i-1}$$

La coordinazione dei delta-time garantisce che le note sovrapposte negli accordi del Pad vengano aperte simultaneamente ($\Delta t = 0$ tra le note dello stesso accordo) e chiuse correttamente a fine battuta. Fatto questo la traccia viene salvata in un file `.mid`. Il file `.mid` generato non serve solo come output finale per l'utente (apribile in qualsiasi DAW o software musicale), ma costituisce l'input diretto per il modulo `synth.py` che legge le 4 tracce MIDI, estrae i messaggi `note_on/note_off`, i canali e le velocità e poi converte ogni nota MIDI nella corrispondente frequenza in Hz ($f = 440 \times 2^{(pitch-69)/12}$) per sintetizzare l'onda audio WAV finale.

Capitolo 4

Conclusioni

4.1 Valutazione finale

La struttura modulare del documento facilita la revisione e la manutenzione del lavoro scientifico. L'inclusione di file separati consente di organizzare in modo razionale teoria, metodo e discussione dei risultati, riducendo il rischio di errori e migliorando la leggibilità complessiva.

4.2 Sintesi

In sintesi, il template proposto unisce la formalizzazione matematica con una presentazione elegante e professionale, utile per relazioni, articoli tecnici e lavori accademici di carattere scientifico.

4.3 Sviluppi futuri

Si potrebbe ampliare all'inglese, spagnolo...

Appendice A

Appendice A

A.1 Esempio di appendice

In appendice si possono inserire dettagli secondari, dimostrazioni o dati supplementari. Ad esempio, la seguente identità è utile in numerosi contesti:

$$\sum_{k=0}^n \binom{n}{k} = 2^n. \tag{A.1}$$

Questa forma può essere usata per riportare risultati accessori senza interrompere il flusso principale dell'articolo.